

Тема 5. АЛГОРИТМИ ЗА ГРАФИЧНО ПРЕДСТАВЯНЕ НА ГРАФИЧНИ ПРИМИТИВИ – ТОЧКА, ЛИНИЯ, ПОЛИГОН

РАБОТА С OPEN GL - ВЪВЕДЕНИЕ, БАЗОВ СИНТАКСИС, БИБЛИОТЕКИ, СТРУКТУРА НА ПРОГРАМА

5.1 РАСТЕРИЗИРАНЕ НА ПРИМИТИВИ

5.2 ИЗЧЕРТАВАНЕ НА ГРАФИЧНИ ПРИМИТИВИ

5.3 OPENGL – ВЪВЕДЕНИЕ, БАЗОВ СИНТАКСИС, БИБЛИОТЕКИ, СТРУКТУРА НА OPEN GL ПРОГРАМА, СЪЗДАВАНЕ НА OPENGL ПРОЗОРЕЦ С GLUT, СЪЗДАВАНЕ НА ГРАФИЧНИ ПРИЛОЖЕНИЯ С БИБЛИОТЕКА OPENGL

упр. 9 OpenGL Работа с графичните примитиви точки, линии

5.1 РАСТЕРИЗИРАНЕ НА ПРИМИТИВИ

Примитиви са най-прости криви, които се използват за чертаене на фигури на екрана като например отсечка, окръжност, дъга от окръжност, елипса, символи.

Под растеризиране на отсечка например, се разбира следното:

Предполагаме, че е зададена отсечка (сегмент) с крайни точки $A = (a_1, a_2)$ и $B = (b_1, b_2)$ дефинирани в правоъгълна координатна система. Целочислените точки, т.е. точките с координати цели числа, ще ги наричаме пиксели. Избирането на онези пиксели, които са най-близо до точките от отсечката се нарича растеризация на отсечката, а избраните пиксели ще ги наричаме растер на отсечката.

Правоъгълникът, в който варират пикселите, се нарича "растерен дисплей" или само "дисплей" или "екран". Това ще част от екрана на монитора. Могат да се използват различни програмни езици за създаване на софтуера за визуализиране на растера.

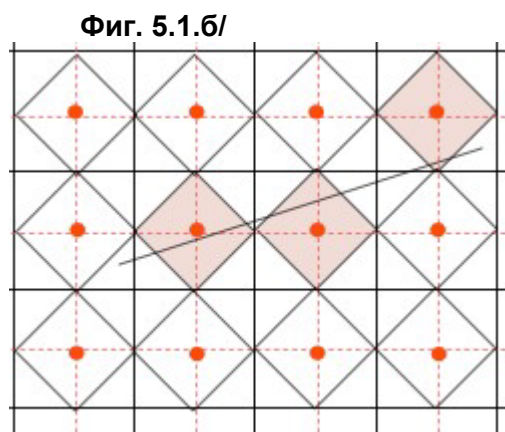
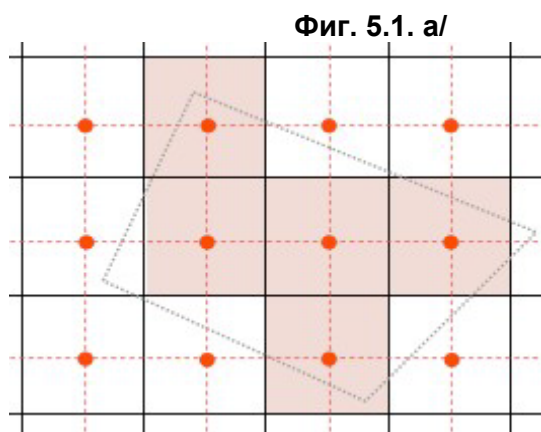
Екранът на този софтуер на дисплея е правоъгълник с размери в зависимост от графичния режим на използвания графичен драйвер.

Алгоритмите за чертаене на отсечка трябва да изпълняват някои изисквания като:

1. Растеризираната отсечка трябва да прилича на отсечка, крайните точки трябва да са избрани акуратно.
2. Осветеността на сегмента трябва да не зависи от мястото и ориентацията.
3. Сегмента трябва да се изчертава бързо.

Правила на растеризацията

- Засветяване на пикселите когато центъра на пиксела е вътре в затворена област. Ако центъра е на контура(ръба) то се засветяват само „левите” и „долните” (Фиг. 5.1.а)
- Засветяване на пикселите когато отсечката пресича фигурата „диамант” (квадрати но получени от пресичане на прави сключващи 45° с координатните оси (Фиг.5.1. б)

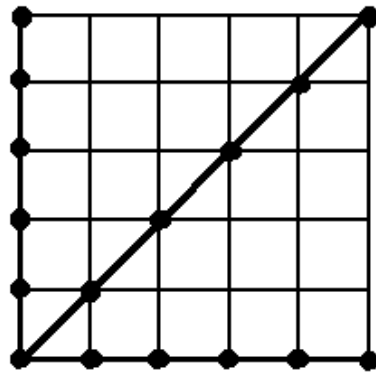


5.2 ИЗЧЕРТАВАНЕ НА ГРАФИЧНИ ПРИМИТИВИ

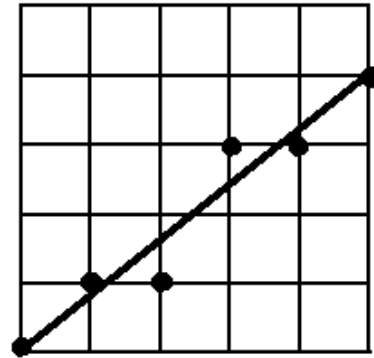
5.2.1 Изчертаване на отсечка. Общи положения

Растровият дисплей може да се разглежда като матрица от пиксели, всеки от които може да бъде засветен. Процесът на определяне на пикселите, апроксимиращи зададена

отсечка най-добре, се нарича разложение на последната в средата на растера. За хоризонтални, вертикални или наклонени под 45° (фиг. 5.2а) линии изборът на нужните елементи е очевиден. За всяка друга тяхна ориентация (фиг. 5.2б) изборът на нужните пиксели е значително по-труден.



Фиг. 5.2 а



Фиг. 5.2 б

В такъв случай, следва да се определят следните най-общи изисквания към алгоритмите за изчертаване на отсечки:

- **отсечките трябва да изглеждат като прави, да започват и завършват в зададени точки.**
- **яркостта във всички точки да е постоянна и да не зависи от наклона.**
- **скоростта на изчертаването им трябва да бъде максимално възможна.**

На практика не всички критерии могат да се удовлетворят напълно и едновременно. Същността на растровия дисплей изключва генерация на напълно прави линии (с изключение на някои частни случаи). Обаче при достатъчно висока разрешаваща способност може да се получи приемливо представяне на идеалните непрекъснати линии с техните еквиваленти, съставени от пиксели с крайна големина.

Постоянна яркост може да се постигне само при вертикални, хоризонтални и наклонени под 45° линии. За всички други ориентации разложението в растера води до неравномерна яркост. Вертикалните и хоризонталните линии изглеждат по-ярки от наклонените, тъй като разстоянието между пикселите е по-малко.

Повечето алгоритми за изчертаване на отсечка използват така наречения **постъпков метод**. Приведените по-долу алгоритми демонстрират този подход.

5.2.2 Алгоритъм за генериране на отсечки по метода на цифровия диференциален анализатор

Един метод за разложение на отсечка в растрното пространство се състои в решаването на едно диференциално уравнение, описващо този процес. В случая, когато неговото решение трябва да бъде именно права линия, видът му е:

$$\frac{dy}{dx} = \text{const.} \quad \text{или} \quad \frac{\Delta y}{\Delta x} = \frac{y_2 - y_1}{x_2 - x_1}$$

Решението се представя във вида:

$$\begin{aligned} y_{i+1} &= y_i + \Delta y \\ y_{i+1} &= y_i + \frac{y_2 - y_1}{x_2 - x_1} \Delta x \end{aligned} \quad (5.1)$$

където (x_1, y_1) и (x_2, y_2) са крайните точки на разлаганата в растрното пространство отсечка, а y_i - началната стойност на поредната стъпка по отсечката. В същност уравнението (5.1) представлява рекурентно съотношение за последователни стойности на y .

Този метод се нарича метод на **Цифров Диференциален Анализатор (ЦДА)** за генериране на отсечка в растровата графика [12], [13]. Тук обикновено Δx или Δy се избират равни на единицата на растера.

Процедура за разлагане в растера на отсечка по метода на ЦДА [4]

Integer - функция за преобразуване на реално число в цяло

Sign - функция, връщаща -1, 0, 1 за съответно отрицателно, нулево и положителна стойност на аргумента

Апроксимира се дължината на отсечката :

if $\text{abs}(x_2 - x_1) > \text{abs}(y_2 - y_1)$ **then**

дължина= $\text{abs}(x_2 - x_1)$

else

дължина= $\text{abs}(y_2 - y_1)$

endif

полага се по-голямото от нарастванията $\Delta x, \Delta y$ равно на единицата на растера.

$\Delta x = (x_2 - x_1) / \text{дължина}$

$\Delta y = (y_2 - y_1) / \text{дължина}$

закръгляват се величините, а не се отделят дробните им части. Използването на знаковата функция прави алгоритъма пригоден за всички квадранти.

$x = x_1 + 0.5 * \text{sign}(\Delta x)$

$y = y_1 + 0.5 * \text{sign}(\Delta y)$

начало на основния цикъл

$i = 1$

while ($i \leq \text{дължина}$)

Plot ($\text{integer}(x), \text{integer}(y)$)

$x = x + \Delta x$

$y = y + \Delta y$

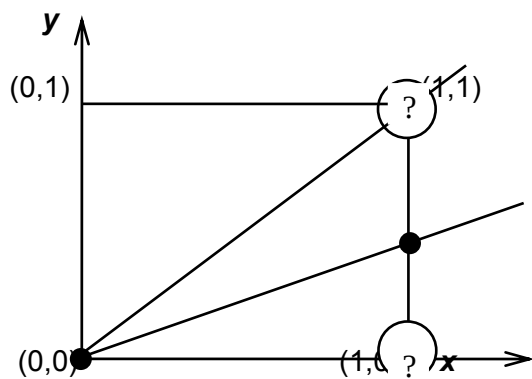
$i = i + 1$

end while

finish

5.2.3 Алгоритъм на Брезенхайм за генериране на отсечки

Този алгоритъм има твърде голямо значение в компютърната графика въобще. Брезенхайм е предложил да се избират **оптимални** в определен смисъл растрови координати на пикселите за представяне на дадена отсечка. В процеса на работа **едната координата - x или y** (в зависимост от ъгловия коефициент на наклона на отсечката) **се изменя с единица. Другата координата се изменя или с 0, или с 1**, в зависимост от разстоянието между действителното положение на отсечката и най-близките координати на мрежата, съставена от центровете на пикселите. Това разстояние се нарича **грешка** и се **отбелязва с ε** . Алгоритъмът на Брезенхайм е конструиран така, че за вземане на решение дали съответната координата да се променя с единица или с нула следва да се проверява **само знакът на грешката**. На фиг 5.3 това се илюстрира за отсечка в първи октант, т.е. за отсечка с ъглов коефициент ($K = \Delta y / \Delta x$) в диапазон от 0 до 1.



Фиг. 5. 3

$$1/2 \leq \Delta y / \Delta x \leq 1 \quad (\text{грешка} \geq 0)$$

$$0 \leq \Delta y / \Delta x < 1/2 \quad (\text{грешка} < 0)$$

Вижда се, че ако ъгловият коефициент на отсечката от точка (0,0) е $> 1/2$, то нейното пресичане с правата $x=1$ ще бъде разположено по-близо до правата $y=1$, отколкото с правата $y=0$. Следователно точката на растера (1,1) по-добре апроксимира хода на отсечката, отколкото точка (1,0). Ако ъгловият коефициент е $< 1/2$, то е вярно обратното. За ъглов коефициент, равен на $1/2$ няма предпочитания при избора. В случая алгоритъмът избира точка (1,1).

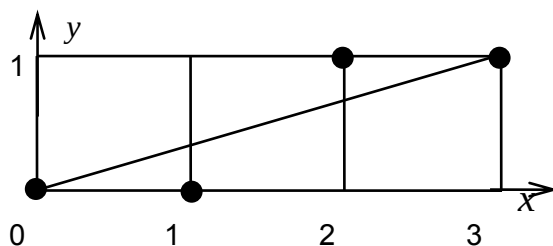
Не всички отсечки преминават през точките на растера. Такава ситуация е представена на фиг 5.3, където отсечка с тангенс на ъгъла на наклона $3/8$ отначало преминава през точка (0,0) и последователно пресича три пиксела. Илюстрирано е изчислението на грешката при представянето на отсечката с дискретни пиксели (фиг 5.3 а).

Тъй като се проверява само знакът на грешката ε , то тя първоначално следва да се установи равна на $-1/2$. По този начин, ако ъгловият коефициент е $\geq 1/2$, големината на грешката в следващата точка на растера (1,0) може да се изчисли така:

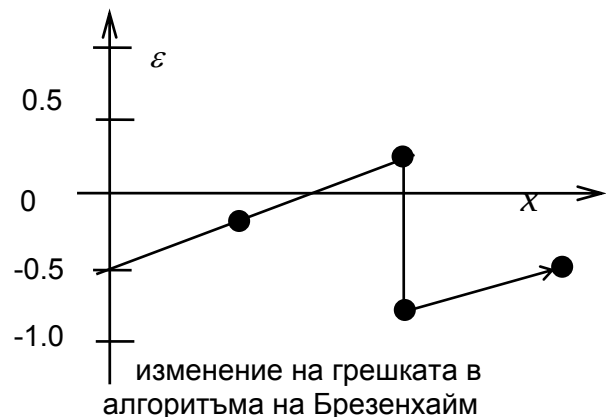
$\varepsilon = \varepsilon + m$, където m е ъглов коефициент.

В случая при начална стойност на грешката равна на $-1/2$

$$\varepsilon = -1/2 + 3/8 = -1/8$$



Фиг. 5. 3



Фиг. 5. 3 а

Тъй като $\varepsilon < 0$, отсечката преминава по-ниско от средата на пиксела. Следователно, пикселът на същото хоризонтално ниво по-добре апроксимира положението на отсечката, затова y не се увеличава. Аналогично се изчислява в следващата точка (2, 0):

$$\varepsilon = -1/8 + 3/8 = 1/4$$

Сега грешката е положителна, (фиг. 5.3а) т.е. отсечката преминава по-високо от средната точка. Точката (2,1) със следваща по големина координата y по-добре апроксимира положението на отсечката. Следователно, **y се увеличава с единица**. Преди да се разгледа следващия пиксел, трябва да се преизчисли грешката, като от нея се **извади единица**, или:

$$\varepsilon = 1/4 - 1 = -3/4$$

Пресичането на вертикалната права $x=2$ със зададената отсечка лежи на $1/4$ по-надолу от правата $y=1$. Ако отсечката се пренесе с $1/2$ по-надолу, то се получава величината $-3/4$. Продължението на изчисленията за следващия пиксел дават:

$$\varepsilon = -3/4 + 3/8 = -3/8$$

Тъй като $\varepsilon < 0$, то y не се увеличава.

От всичко това следва, че **грешката - това е интервалът, отсичан по оста y на разглежданата отсечка във всеки растрров елемент (относно $-1/2$).**

Алгоритъм на Брезенхайм за първи октант ($0 \leq \Delta y \leq \Delta x$) [4]

$x, y, \Delta x, \Delta y$ - цели, ε - реално

инициализация на променливите

$$x = x_1$$

$$y = y_1$$

$$\Delta x = x_2 - x_1$$

$$\Delta y = y_2 - y_1$$

$$\varepsilon = \Delta y / \Delta x - 1/2 \text{ - инициализация на } \varepsilon \text{ с поправка от половин пиксел}$$

начало на основния цикъл

```
for i=1 to  $\Delta x$ 
  Plot (x, y)
  while ( $\varepsilon \geq 0$ )
    y=y+1
     $\varepsilon = \varepsilon - 1$ 
  end while
  x=x+1
   $\varepsilon = \varepsilon + \Delta y / \Delta x$ 
next i
finish
```

В този си вид алгоритъмът на Брезенхайм предполага използване на аритметика с плаваща запетая и деление. Бързодействието на алгоритъма може да се увеличи с използване на целочислена аритметика и изключвайки делението. Тъй като е важен само знакът на грешката, то преобразуването:

$$\bar{\varepsilon} = 2 \cdot \varepsilon \cdot \Delta x$$

превръща горния алгоритъм в целочислен и позволява ефективното му реализиране на апаратно или микро-програмно ниво.

За пълна реализация на алгоритъма на Брезенхайм е необходимо да се обработват отсечките във всички октанти. Модификацията може да се направи, като се отчита номера на квадранта, в който лежи отсечката и ъгловия коефициент. Когато абсолютната стойност на ъгловия коефициент е >1 , y постоянно се изменя с единица, а критерият за грешката на Брезенхайм се използва за вземане на решение за изменение на x. Изборът на постоянно изменящи се (с +1 или -1) координати зависи от квадранта.

Обобщен целочислен алгоритъм на Брезенхайм за всички квадранти [11]

```
x = x1
y = y1
 $\Delta x = \text{abs}(x_2 - x_1)$ 
 $\Delta y = \text{abs}(y_2 - y_1)$ 
 $s_1 = \text{Sign}(x_2 - x_1)$ 
 $s_2 = \text{Sign}(y_2 - y_1)$ 
```

```
if  $\Delta y > \Delta x$  then
```

```
  Temp= $\Delta x$ 
   $\Delta x = \Delta y$ 
   $\Delta y = \text{Temp}$ 
  Obmen=1
```

```
else
```

```
  Obmen=0
```

```
endif
```

$$\bar{\varepsilon} = 2 * \Delta y - \Delta x$$

основен цикъл

```
for i=1 to  $\Delta x$ 
```

```
  Plot (x, y)
```

```
  while ( $\bar{\varepsilon} \geq 0$ )
```

```
    if Obmen = 1 then
```

$$x = x + s_1$$

```
    else
```

$$y = y + s_2$$

```
    endif
```

$$\bar{\varepsilon} = \bar{\varepsilon} - 2 * \Delta x$$

```
  end while
```

```
  if Obmen=1 then
```

$$y = y + s_2$$

```

else
     $x = x + s,$ 
endif
 $\bar{\varepsilon} = \bar{\varepsilon} + 2 * \Delta y$ 
next i
finish

```

5.3 OpenGL – Въведение, базов синтаксис, библиотеки, структура на OpenGL програма, създаване на OpenGL прозорец с GLUT, създаване на графични приложения с библиотека OpenGL

OpenGL – Въведение [използвани материали от Иван Георгиев Иванов]

OpenGL е безплатна библиотека, много-платформена (т.е. не е приложима само под Windows, но и под останалите операционни системи). Тя е гъвкаво средство за създаване на професионална **3D** графика, която може да бъде използвана за много цели. Библиотеката е създадена от компанията **SiliconGraphics**. Още от началото си **OpenGL** се развива непрекъснато. В процеса на увеличаване функционалността на **OpenGL** взимат участие много компании, т.н. консорциум **OpenGL Architecture Review Board (ARB)**. Създадени са много библиотеки и допълнителни модули (т.н. **OpenGL extensions**), някои от които улесняват допълнително работата с **OpenGL**, а други осъществяват връзката между **OpenGL** и най-новите технологии за **3D** графика в модерните видео-ускорители. С последната си версия **2.0** издадена официално от **ARB** на 7 септември 2004г, **OpenGL** показва че няма намерение да отстъпи на своя пряк конкурент **DirectX**, създаден от **Microsoft**. Сам по себе **OpenGL** не включва функции за създаването на прозорец, в който ще бъде визуализирана създадената от вас **3D** графика, т.е. не можете да контактувате директно с операционна система, не можете да следите за натиснат бутон от клавиатурата, няма готови функции за изрисуване на сложни обекти, както и за пресъздаване на ефекти като пушек, огън. Всичко това (или почти всичко) може да бъде извършено с някоя от помощните библиотеки към **OpenGL**, за които стана въпрос вече. Ще се разгледат главно библиотеката **GLUT** (другите **GLU**, **GLAUX**), която има завидна функционалност за построяването на **OpenGL** прозорец. Ще се покаже как да се построи **OpenGL** прозорец, като се използват и **Win32API** на **Microsoft** за **Windows**, което е по-трудно, но и по-добро решение за мащабни, **Windows** ориентирани проекти.

Всяка функция, която принадлежи към **OpenGL** библиотеката започва с "gl" (например: **glRotate**). Функциите от **GLU** започват с "glu", от **GLAUX** със "aux", а от **GLUT** съответно със "glut". Така се определя коя библиотека трябва да се включи, когато се използват определени функции.

Също така някои функции от **OpenGL** завършват накрая с буква или с число и буква, което обозначава колко и от какъв тип аргументи приема съответната функция. Например: **f** - **float**, **d** - **double** , **i** - **int**. Наставка като **3f** означава, че функцията приема 3 аргумента от тип **float**. Например: **glVertex3f()**. Някои функции могат да завършват с една от буквите " f ", " d " или " i " заедно с буквата " v ". Например наставка като " fv ", означава че функцията приема указател към масив от тип **float**.

Голяма част от функциите имат по няколко разновидности, всяка от които се различава от другите по броя и типа на стойностите, които приемат. Например: **function{234}{fd}()** - тази функция може да приеме 2, 3 или 4 аргумента от тип **float** или **double**. В този случай, ако искате да подадете 2 аргумента от тип **float**, ще трябва да използвате следния вариант: **function2f()**.

В **OpenGL** можете да се ползват стандартните типове данни, както в **C/C++**, но също така може да ползвате и вградените в **OpenGL** типове данни, които отговарят съответно на :

GLbyte = signed char

GLubyte / GLboolean = unsigned char

GLshort = short int

GLushort = unsigned short int

GLint / GLsizei = long

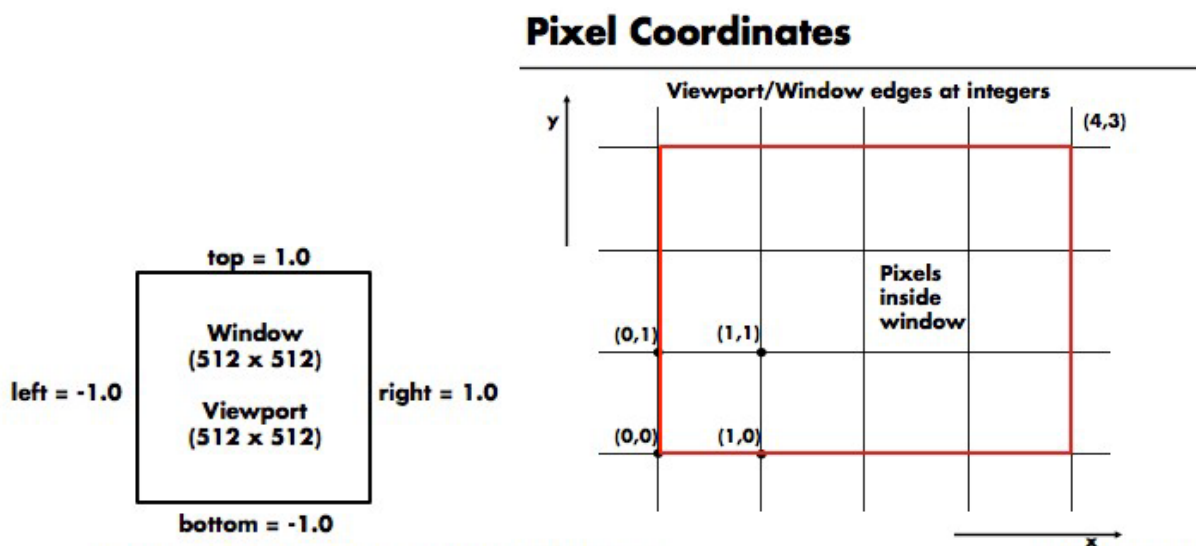
GLuint / GLenum / GLbitfield = unsigned long

GLfloat / GLclampf = float

GLdouble / GLclampd = double

В примерните сорс кодове е използван безплатния компилатор **Microsoft Visual C++ Learning Edition**, а при голяма част от тях за връзка с операционната система е използвана библиотеката **GLUT**, която е изключително лесна за усвояване, но примерите за напреднали използват **Win32 API**. Примерните сорс кодове ще бъдат предназначени за **Microsoft Windows**, като най-широко разпространената ОС. Разбира се, кода може да бъде прекомпилиран и за други операционни системи, като използваме версията на **GLUT** за други ОС.

Графичен Прозорец и координати в OpenGL.



Viewport прозореца е част и може да е по малък от прозореца Window, което ще промени и координатите в двата прозореца.

Създаване на OpenGL прозорец с GLUT

За построяването на прозорец, в който да се визуализира графиката се използва библиотеката GLUT, чиято последна версия може да намерите на официалния сайт на OpenGL - www.opengl.org

Заедно с повечето компилатори се предлага библиотеката GLAUX и макар че тя е близка на GLUT, втората притежава значително по-голяма функционалност. Библиотеката се състои от три файла - glut32.dll, glut32.lib и glut.h, като всеки от тях трябва да бъде разположен на правилното място:

glut32.dll - в папката Windows,

glut32.lib - съответно при останалите *.lib файлове в папката LIB на компилатора, най - вероятно това е Microsoft Visual C++. (C:/ Microsoft Visual Studio / VC98 / LIB)

glut32.h - съответно при останалите хедърните файлове в папката Include, но най-добре в под-папката GL. Директория: C:/ Microsoft Visual Studio / VC98 / Include / GL

При построяването на проекта, се избира Win32 Console Application, а не Win32 Application. След като е готов проекта трябва да се свържат библиотеките : Project / Settings / Link / във реда Object/Library Modules пред kernel32.lib трябва да добавите файловете opengl32.lib и glut32.lib

Структура на OpenGL програма

Първо трябва да се включат хедърните файлове. Въпреки че се използват GLUT за създаването на прозореца, той взаимодейства с Windows, така че трябва да се включи и хедърния файл windows.h

```
#include < windows.h > // Хедърният файл за Windows
```

```
#include < gl/glut.h > // Хедърният файл за GLUT
```

```
#include < gl/gl.h > // Хедърният файл за OpenGL
```

```
void Render( ); // Прототипът на рендериращата ни функция
```

Функцията main(), която приема целочисления аргумент argc и масива от низови указатели *argv[]

```
int main( int argc, char *argv[ ] )
```

```
{
```

```
    glutInit( &argc, argv ); // 1
```

```
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_DEPTH | GLUT_RGBA); // 2
```

```
    glutInitWindowSize( 400 , 400 ); // 400 пиксела височина и широчина за прозореца // 3
```

```
    glutCreateWindow( "Created with GLUT (OpenGL Utility Toolkit)" ); // 4
```

```
    glutDisplayFunc( Render ); // 5
```

```
    glutMainLoop( ); //6
```

```
    return 0;
```

```
}
```

1. Инициализиране библиотеката GLUT с функцията: **glutInit(int *argc, char *argv[])**

2. Определяне на режима на представяне с функцията **glutInitDisplayMode**, която приема няколко аргумента разделени със символа " | ". Това са :

- **GLUT_SINGLE** или **GLUT_DOUBLE** за единичен или двоен буфер. За предпочитане е използването на двоен буфер, освен ако няма да се представя някоя 2D фигура. За триизмерна графика е задължително използването на двоен буфер, който е и много по-бърз. При **GLUT_DOUBLE** само единия буфер е видим, а в другия се създава графиката и след това двата буфера се разменят.

- **GLUT_DEPTH**, **GLUT_STENCIL** и/или **GLUT_ACCUM**. Първата константа определя наличието на z-буфер. За триизмерна графика неговото включване е задължително.

- **GLUT_RGBA** или **GLUT_INDEX** определя начина за изграждане на цветовете във вашата програма. При GLUT_INDEX цвета се определя от т.н. цвetoва палитра, която може да съдържа 16 или 256 цвята и на всяко число от 0 до 15 или от 0 до 255 съответства точно определен цвят - безспорно остарял стандарт, който дори възпрепятства използването на други ефекти като например - динамична светлина. За предпочитане е използването на GLUT_RGBA при който цвета се сформира от количеството на RED, GREEN, BLUE цветовете и ALPHA канала (полу-прозрачност).

3. Определяне големината на прозореца с функцията: **glutInitWindowSize(int width, int height)**

4. Създаване на прозореца с функцията: **glutCreateWindow(char *string)**, която приема избраното име за прозореца.

5. Извикване на функцията **glutDisplayFunc(void (*func)(void))**, която извиква на свой ред рендериращата функция всеки път когато прозореца трябва да бъде пре-рисуван. Това може да се случи когато прозореца се показва за първи път, когато е повреден или ако се извика функцията glutPostRedisplay().

6. Извикване функцията **glutMainLoop()**, която стартира цикъла на съобщенията.

Във функцията **main** могат да се включат функции от GLUT, които следят за съобщения от мишката и клавиатурата, оразмеряват правилно прозореца при минимизиране и максимизиране и много други.

Рендерираща функция:

```
void Render() {  
glClear( GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT ); // 7  
void glClearColor( GLclampf red, GLclampf green, GLclampf blue, GLclampf alpha ) - // 8 тя  
приема RGBA цвета на фона. С тази функция може да създадете реалистична мъгла  
OpenGL код  
glutSwapBuffers( ); // 9  
glutPostRedisplay( ); //10  
}
```

7. Първо се изчиства буфера от предишното изображение. Това става с функцията **glClear()**, която вече е част от OpenGL. Тя може да приема няколко от следните аргументи разделени със знака " | " : **GL_COLOR_BUFFER_BIT** - изчиства цветовия буфер; **GL_DEPTH_BUFFER_BIT** - изчиства z – буфера; **GL_ACCUM_BUFFER_BIT** - изчиства Accumulation (натрупващия) буфера; **GL_STENCIL_BUFFER_BIT** - изчиства Stencil (шаблон) буфера;

8. С функцията **glClearColor()**, може да се запълни цветовия буфер с определен цвят или по-просто казано - да определите цвета на фона. Нейният прототип е:
void glClearColor(GLclampf red, GLclampf green, GLclampf blue, GLclampf alpha) - тя приема RGBA цвета на фона. Тази функция се използва за създаване на реалистична мъгла.

9. Ако се използва двоен буфер, задължително трябва да се извика функцията **glutSwapBuffers()**, която разменя двата буфера т.е невидимия буфер в който изображенията са вече създадени се показва, а буфера който до момента е бил видим става невидим, изчиства се и в него се създава следващия кадър, който трябва да бъде показан в последствие. В случай, че предпочетете единичен буфер, трябва да извикате функцията **glFlush()**, вместо **glutSwapBuffers()**.

10. Функцията **glutPostRedisplay()** съобщава на GLUT, че прозореца трябва да бъде прерисуван. Това води до извикване на **glutDisplayFunc()**, която пък извиква рендериращата ни функция. Така се стига до един безкраен цикъл, който обаче е задължителен за да не показва програмата само първия кадър от 3D графиката и да спира. След компилиране на този сорс код и стартиране на програмата ще се създаде вашия прозорец показващ само черен фон, заедно с още един DOS прозорец.

Често използвани функции от библиотеката GLUT.

int glutCreateWindow(char *name) - създава прозорец, но и връща неговия идентификационен номер. Например: win = glutCreateWindow("Window") , където win е променлива от тип int.

glutDestroyWindow(int win) - унищожава прозорец, определен от идентификационния номер win.

int glutGetWindow(void) - връща идентификационния номер на текущия прозорец.

glutInitWindowPosition(int x, int y) - определя началното местоположение на прозореца като приема два аргумента (x и y координата), които се отнасят за горния ляв ъгъл на прозореца. При операционна система Windows най-горната лява точка от екрана на монитора има координати (0 , 0). Специално функцията glutInitWindowPosition() трябва да бъде извикана преди функцията glutCreateWindow(), която изгражда прозореца.

glutPositionWindow(int x, int y) - променя позицията на създадения вече прозорец в зависимост от подадените аргументи.

glutReshapeWindow(int width, int height) - променя размерите на създадения прозорец в зависимост от подадените аргументи.

glutSetWindowTitle(char *name) - определя/променя заглавието на прозореца в зависимост от подадения аргумент.

Следващите функции трябва да бъдат извикани задължително след създаването на прозореца с glutCreateWindow и преди започване на цикъла на съобщенията т.е. преди функцията **glutMainLoop()**.

glutFullScreen(void) - оразмерява прозореца на цял екран.

glutReshapeFunc(void (*func)(int width, int height)) - определя функцията, която се извиква когато прозореца промени своя размер. На потребителската функция се подават новите размери на прозореца. Например : **glutReshapeFunc(OurFunction)**, където потребителската функция трябва да е от вида **void OurFunction(int w, int h)**

glutSetCursor(int cursor) - променя курсора на мишката в зависимост от подадения аргумент, който може да бъде една от константите :

GLUT_CURSOR_RIGHT_ARROW - наклонена надясно стрелка

GLUT_CURSOR_LEFT_ARROW - наклонена наляво стрелка

GLUT_CURSOR_INFO - сочеща ръка

GLUT_CURSOR_DESTROY - череп с кръстосани кости

GLUT_CURSOR_HELP - стрелка с въпросителен знак

GLUT_CURSOR_CYCLE - кръг с пресечена черта

GLUT_CURSOR_SPRAY - спрей

GLUT_CURSOR_WAIT - пясъчен часовник

GLUT_CURSOR_TEXT - знака " I ", когато добавяте текст

GLUT_CURSOR_CROSSHAIR - кръст

GLUT_CURSOR_UP_DOWN - две противоположно сочещи вертикални стрелки, свързани в една цяла - появява се когато искате да промените размера на даден прозорец

GLUT_CURSOR_LEFT_RIGHT - две противоположно сочещи хоризонтални стрелки, свързани в една цяла

GLUT_CURSOR_TOP_SIDE - стрелка сочеща нагоре

GLUT_CURSOR_BOTTOM_SIDE - стрелка сочеща надолу

GLUT_CURSOR_LEFT_SIDE - стрелка сочеща наляво

GLUT_CURSOR_RIGHT_SIDE - стрелка сочеща надясно

GLUT_CURSOR_TOP_LEFT_CORNER - стрелка сочеща към горния ляв ъгъл

GLUT_CURSOR_TOP_RIGHT_CORNER - стрелка сочеща към горния десен ъгъл

GLUT_CURSOR_BOTTOM_RIGHT_CORNER - стрелка сочеща към долния десен ъгъл

GLUT_CURSOR_BOTTOM_LEFT_CORNER - стрелка сочеща към долния ляв ъгъл

GLUT_CURSOR_NONE - невидим курсор

Обработка на събития :

glutKeyboardFunc(void (*func)(unsigned char key,int x, int y)) - определя коя функция да бъде извикана при натискане на клавиш от клавиатурата.Тя трябва да е от вида void OurFunction(unsigned char key, int x, int y). На потребителската функция се предава ASCII кода на натиснатия клавиш и координатите на мишката.

glutSpecialFunc(void (*func)(int key, int x, int y)) - определя коя функция да бъде извикана ако бъде натиснат някой от специалните клавиши на клавиатурата.Тя трябва да е от вида void OurFunction(int key, int x, int y). Първия аргумент може да бъде една от константите : GLUT_KEY_F1, ..., GLUT_KEY_F12, GLUT_KEY_LEFT, GLUT_KEY_RIGHT, GLUT_KEY_UP, GLUT_KEY_DOWN, GLUT_KEY_PAGE_UP, GLUT_KEY_PAGE_DOWN, GLUT_KEY_INSERT, GLUT_KEY_END. Другите два аргумента определят местоположението на курсора на мишката.

glutMouseFunc(void (*func)(int button, int state, int x, int y)) - определя функцията, която се извиква при натискане/отпускане на бутон на мишката. Тя трябва да е от вида void OurFunction(int button, int state, int x, int y). Първият от подадените аргументи може да е равен на една от константите : GLUT_LEFT_BUTTON, GLUT_RIGHT_BUTTON и GLUT_MIDDLE_BUTTON. Втория аргумент (state) може да е равен на GLUT_DOWN ако бутона е натиснат или на GLUT_UP ако бутона е отпуснат. Последните два аргумента определят местоположението на курсора на мишката.

glutMotionFunc(void (*func)(int x, int y)) - определя функция, която се извиква ако мишката се премести докато някой от бутоните и е натиснат. Двата аргумента определят местоположението на мишката.

glutPassiveMotionFunc(void (*func)(int x, int y)) - определя функция, която се извиква, ако мишката се премести докато не е натиснат никой от бутоните ѝ.

glutEntryFunc(void (*func)(int state)) - определя функция, която се извиква когато курсора на мишката напусне прозореца или влезе отново в него. Подаденият аргумент може да бъде една от двете константи GLUT_LEFT(ако курсора излиза от прозореца) или GLUT_ENTERED (ако курсора влиза в прозореца).

glutIdleFunc(void (*func)(void)) - определя функция, която се извиква ако няма възникнали събития т.е ако никоя от по-рано изброените функции не бъде извикана.

glutTimerFunc(unsigned int msec, void (*func)(int value), int value) - определя функция, която се извиква след определено време. Първият аргумент (msec) определя интервала от време в милисекунди, а последния аргумент (value) се подава на потребителската функция, която се извиква от glutTimerFunc(). Тя трябва да бъде от вида void OurFunction(int value).

Създаване на менюта :

Библиотеката GLUT предлага няколко функции, с които бързо и лесно могат да се създадат различни контекстни менюта. Функциите, с които се създава меню са :

int glutCreateMenu(void (*func)(int value)) - тя връща идентификационния номер на създаденото меню, който трябва да е от тип int и определя функция, която трябва да бъде извикана при избиране на елемент от менюто. Функцията трябва да е от вида void OurFunction(int value). Подаденият ѝ аргумент е идентификационния номер на избрания елемент от менюто.

void glutSetMenu(int menu) - определя текущо контекстно меню, в зависимост от подадения идентификационен номер.

int glutGetMenu(void) - връща идентификационния номер на текущото контекстно меню.

void glutDestroyMenu(int menu) - унищожава контекстното меню, чиито идентификационен номер сме подали като аргумент на функцията.

void glutAddMenuEntry(char *name, int value) добавя елемент към текущото контекстно меню. Първият аргумент определя името на елемента, което ще видим в менюто, а вторият идентификационния номер на елемента. Обикновено първият елемент има иден. номер 1, втория - 2 и т.н.

void glutAddSubMenu(char *name, int menu) - добавя подменю към текущото контекстно меню. Първият аргумент определя текста, който ще се покаже в текущото меню, от който ще излиза подменюто при задържане на курсора на мишката върху него. Втория аргумент приема идентификационния номер на вече създадено меню, което да се превърне в подменю на текущото меню.

void glutRemoveMenuItem(int entry) - премахва елемент от менюто с пореден номер entry. Първият елемент има пореден номер 1. Подменютата са също елементи на главното меню, така че и те имат поредни номера и могат да бъдат премахвани.

void glutAttachMenu(int button) - определя с кой бутон на мишката да се пуска определено контекстно меню. Аргумента button може да бъде една от константите GLUT_LEFT_BUTTON, GLUT_MIDDLE_BUTTON или GLUT_RIGHT_BUTTON.

void glutDetachMenu(int button) - премахва възможността на дадено меню да бъде пуснато със съответния бутон, определен от аргумента button.

Начин на използване Преди да се създаде меню, трябва да се дефинират променлива от тип int, която ще се използва за съхранение на идентификационния номер на менюто. След това се създава менюто : ID = glutCreateMenu(OurFunction), където ID е променлива. След това трябва да се определи това меню като текущо чрез извикване на glutSetMenu(ID). Вече може да се добавят елементи с glutAddMenuEntry() или подменютата с glutAddSubMenu(). Накрая се извиква функцията glutAttachMenu(int button) за да се определи с кой бутон на мишката да се пуска менюто. Може да има три действащи менюта, които да се пускат с трите бутона на мишката, като в този случай се извършват гореописаните стъпки за всяко меню поотделно.

6.2 OPENGL - ПРОСТИ ОБЕКТИ

OpenGL предоставя функции за изрисване само на прости обекти - линии, триъгълници, полигони, които могат да бъдат свързани в някоя по-сложна фигура. Разбира се и библиотеките **GLAUX** и **GLUT** правят лесно създаването на често използвани фигури като сфера, куб и различни многоъгълници. Библиотеката **GLU** дори представя функции, с които може да създаде иначе доста сложни обекти като цилиндър, диск, криви, релеф.

За да се създаде проста фигура трябва да дефинирате нейните точки (т.е. нейните върхове „vertex“). За всеки един връх може да се определи цвят, полу-прозрачност и нормала (вектор, който се използва за определянето на динамичната светлина, която попада върху съответната повърхност). Определянето на върховете на даден обект трябва да стои между функциите **glBegin(GLenum Mode)** и **glEnd()**. Те показват началото и края на дадено изрисване. За всеки връх се използва текущия цвят, който може да се промени при повторно извикване на определена функция но с различни аргументи.

Определянето на координатите на точка става с функцията :

glVertex{234}f[v](x, y, z, (w)) - функцията може да приема от два до четири аргумента, а също и масив от стойности. За определяне на някаква точка в **3D** пространството се извиква : **glVertex3f(x, y, z)** - аргументът w е тегловния коефициент, (по подразбиране равен на 1) и на него се делят еднородните координати x , y и z .

Определянето на цвят в **RGBA** режим става с функцията:

glColor{34}f[v](R, G, B, (A)) - цвета се изгражда в зависимост от наситеността на червения, синия и зеления цвят, която се определя в части от цялото [0;1]. Ако искате наполовина наситено червено трябва да извикате функцията със следните параметри : **glColor3f(0.5 , 0.0 , 0.0);**

Четвъртият аргумент, който не е задължителен е прозрачността. Тя обаче трябва да се зададе предварително с други функции. Полу-прозрачността се разглежда в друга тема. Стойността 1.0 задава че няма полу-прозрачност.

При **Color-Index** режим функцията за определяне на цвят е :

glIndex(index) - като число от 0 - 255 определя различен цвят.

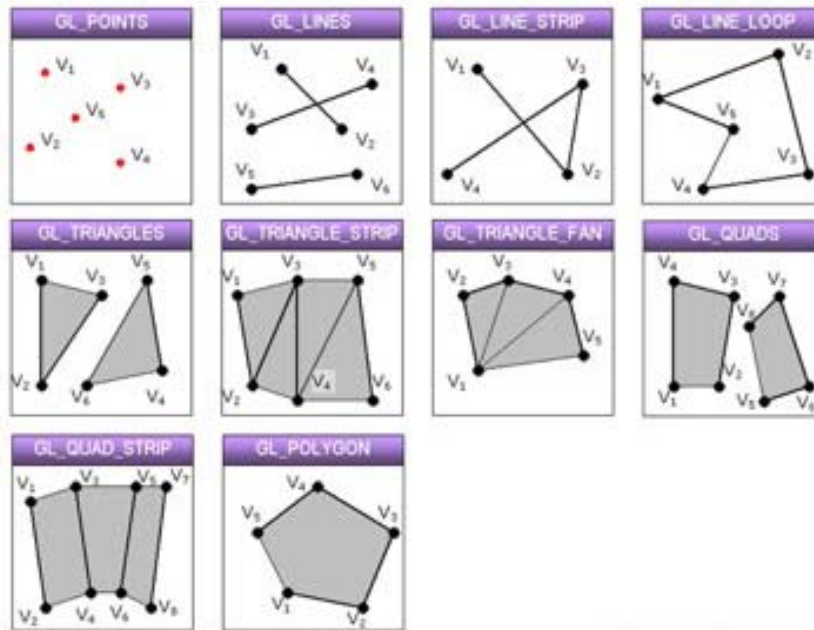
Може да се определи различен цвят за всеки връх и така фигурата ще прелива в зависимост от цвета на нейните върхове.

С функцията **glShadeModel(GLenum mode)** се определя начина на определяне на цвета. Тя може да приеме една от константите:

GL_SMOOTH - за преливащ цвят, което е по-подразбиране.

GL_FLAT - за едноцветни фигури, независимо от цвета на върховете.

Преди да изрисувате някакъв обект трябва да се подадат като аргумент на функцията `glBegin( GLenum mode )` една от следните константи:



GL_POINTS - остават отделни точки.

GL_LINES - точките се свързват две по две в отделни линии. Ако броя на точките е нечетен, последната точка остава несвързана.

GL_LINE_STRIP - всяка точка се свързва със следващата.

GL_LINE_LOOP - всяка точка се свързва със следващата, като последната се свързва с първата.

GL_POINTS, GL_LINES, GL_LINE_STRIP, GL_LINE_LOOP, GL_TRIANGLES, GL_TRIANGLE_STRIP, GL_TRIANGLE_FAN, GL_QUADS, GL_QUAD_STRIP, GL_POLYGON

GL_TRIANGLES - точките се свързват в отделни триъгълници. Ако останат една или две точки те не се свързват.

GL_TRIANGLE_STRIP - точките се свързват в серия от триъгълници. Първо се свързват точки 1, 2 и 3, след това 3, 2 и 4, следват 3, 4 и 5, след това 4, 5 и 6 и т.н.

GL_TRIANGLE_FAN - точките се свързват в отново в серия от триъгълници, но начина на свързване е различен. Първо 1, 2 и 3 после 1, 3 и 4 след това 1, 4 и 5 и т.н.

Така се получава някакво подобие на диск защото всички триъгълници имат една допирна точка.

GL_QUADS - точките се свързват в отделни четириъгълници. Ако броя на точките не е кратен на 4, последните остават несвързани.

GL_QUAD_STRIP - точките се свързват в серия от четириъгълници, като първо се свързват точки 1, 2, 4 и 3 следват 3, 4, 6 и 5, след това 5, 6, 8 и 7 и т.н.

GL_POLYGON - точките се свързват в полигон, който трябва да е задължително изпъкнал, иначе са възможни аномалии.

Функции от високо ниво, които създават наготово някои по-сложни обекти :

glRectf(x1, x2, y1, y2) - тази функция изгражда правоъгълник. Съответно (x1, y1) са координатите на горния ляв ъгъл, а (x2, y2) на долния десен ъгъл.